Compiler Design Lab Programs

1. **The lexical analyser should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Develop a lexical Analyzer to identify identifiers, constants, operators using C program.**

Code:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int main() {
    char str[100];
    char identifiers[50][20], constants[50][20], operators[50];
    int i, j, idCount = 0, constCount = 0, opCount = 0;
    printf("Enter the string: ");
    fgets(str, sizeof(str), stdin);
    for (i = 0; str[i] != '\0'; i++) {
        if (isalpha(str[i]) || str[i] == '_') {
            j = 0;
            char temp[20];
            while (isalnum(str[i]) || str[i] == '_')
                temp[j++] = str[i++];
            temp[j] = '\0';
            strcpy(identifiers[idCount++], temp);
            i--;
        }
        else if (isdigit(str[i])) {
            j = 0;
            char temp[20];
            while (isdigit(str[i]))
                temp[j++] = str[i++];
            temp[j] = '\0';
```

```
            strcpy(constants[constCount++], temp);
            i--;
        }
        else if (str[i] == '+' || str[i] == '-' || str[i] == '*' || str[i] == '/' || str[i] == '=') {
            operators[opCount++] = str[i];
        }
    }
    printf("\nIdentifiers: ");
    for (i = 0; i < idCount; i++)
        printf("%s ", identifiers[i]);
    printf("\nConstants: ");
    for (i = 0; i < constCount; i++)
        printf("%s ", constants[i]);
    printf("\nOperators: ");
    for (i = 0; i < opCount; i++)
        printf("%c ", operators[i]);
    printf("\n");
    return 0;
}
```

Output:

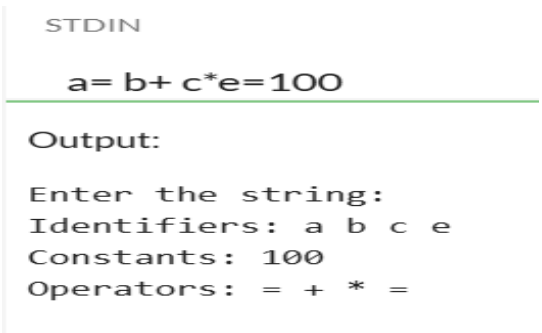
Enter the string: a=b+c*e+100

Identifiers: a b c e

Constants: 100

Operators: = + * +

Output Screenshot:



2. Extend the lexical Analyzer to Check comments, dened as follows in C:

a) A comment begins with // and includes all characters until the end of that line.
b) A comment begins with /* and includes all characters through the next occurrence of the character sequence */Develop a lexical Analyzer to identify whether a given line is a comment or not.

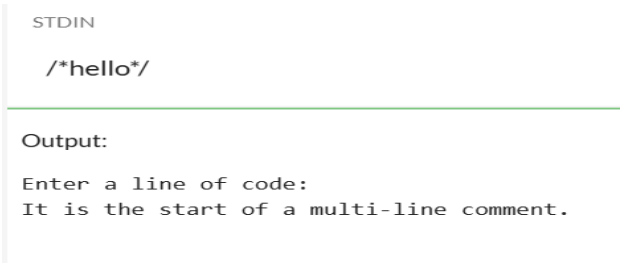
Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char line[100];
    printf("Enter a line of code:\n");
    fgets(line, sizeof(line), stdin);
    if (line[0] == '/' && line[1] == '/') {
        printf("It is a single-line comment.\n");
    }
    else if (line[0] == '/' && line[1] == '*') {
        int len = strlen(line);
        if (len >= 4 && line[len - 3] == '*' && line[len - 2] == '/') {
            printf("It is a multi-line comment.\n");
        } else {
            printf("It is the start of a multi-line comment.\n");
        }
    }
    else {
        printf("It is NOT a comment.\n");
    }
    return 0;
}
```

Output:

```
Enter a line of code: /*hello*/
It is the start of a multi-line comment.
```

Output Screenshot:



3. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular Arithmetic operators.

Code:

```
#include <stdio.h>
int main() {
    char ch;
    printf("Enter an operator: ");
    scanf("%c", &ch);
    switch (ch) {
        case '+':
            printf("Valid operator: Addition (+)\n");
            break;
        case '-':
            printf("Valid operator: Subtraction (-)\n");
            break;
        case '*':
            printf("Valid operator: Multiplication (*)\n");
            break;
        case '/':
            printf("Valid operator: Division (/)\n");
            break;
        default:
            printf("Invalid operator!\n");
    }
    return 0;
```

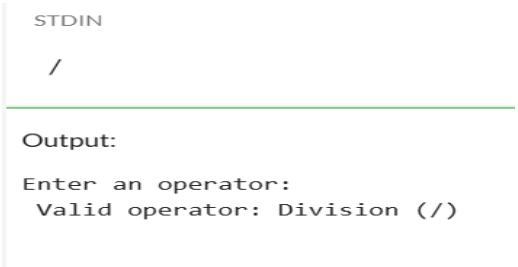
}

Output:

Enter an operator: /

Valid operator: Division (/)

Output Screenshot:



4. Design a lexical Analyzer to find the number of whitespaces and newline characters.

Code:

```
#include <stdio.h>
int main() {
    char ch;
    int space_count = 0, tab_count = 0, newline_count = 0;
    printf("Enter text (press Ctrl+D to end input on Linux/Mac or Ctrl+Z on Windows):\n");
    while ((ch = getchar()) != EOF) {
        if (ch == ' ')
            space_count++;
        else if (ch == '\t')
            tab_count++;
        else if (ch == '\n')
            newline_count++;
    }
    printf("\nNumber of spaces: %d\n", space_count);
    printf("Number of tabs: %d\n", tab_count);
    printf("Number of newlines: %d\n", newline_count);
    return 0;
}
```

Output:

Enter text: Hello Computer.

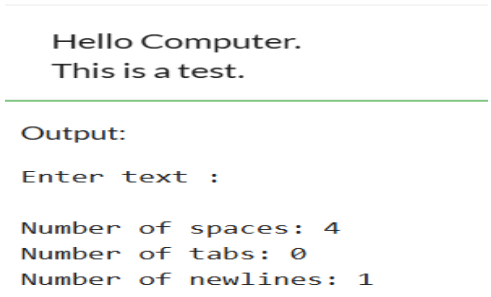
This is a test.

Number of spaces: 4

Number of tabs: 0

Number of newlines: 1

Output Screenshot:



5. Develop a lexical Analyzer to test whether a given identifier is valid or not.

Code:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int isKeyword(char str[]) {
   char keywords[][10] = {"int", "float", "char", "double", "if", "else",  "while", "for", "return", "void", "break", "continue"};
   int n = sizeof(keywords)/sizeof(keywords[0]);
   for(int i = 0; i < n; i++) {
      if(strcmp(str, keywords[i]) == 0)
         return 1;
   }
   return 0;
}
int main() {
   char identifier[50];
   int valid = 1;
```

```
    printf("Enter an identifier: ");
    scanf("%s", identifier);
    if (!(isalpha(identifier[0]) || identifier[0] == '_')) {
        valid = 0;
    } else {
        for(int i = 1; identifier[i] != '\0'; i++) {
            if (!(isalnum(identifier[i]) || identifier[i] == '_')) {
                valid = 0;
                break;
            }
        }
    }
    if (isKeyword(identifier)) {
        valid = 0;
    }
    if(valid)
        printf("'%s' is a valid identifier.\n", identifier);
    else
        printf("'%s' is NOT a valid identifier.\n", identifier);
    return 0;
}
```

Output:

Enter an identifier: myVar1

'myVar1' is a valid identifier.

Output Screenshot: